UNIVERSIDAD — FACULTAD DE INGENIERÍA

PROGRAMA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

INFORME TÉCNICO: SIMULACIÓN DE ECOSISTEMA DISCRETO EN MEMORIA COMPARTIDA

Modelado Predator-Prey en C++17, Paralelismo con OpenMP, Sincronización Lock-Free y Visualización Web

Asignatura: Computación Paralela y Distribuida
Semestre Académico: 10° Semestre
Estudiante: Steban Rivera
Repositorio Oficial: https://github.com/Steban-Rivera/Microproyecto-3-Ecosystem-Simulation
Tecnologías del Proyecto: C++17, OpenMP 5.2, CMake, GCC / MinGW-w64, JavaScript (Web)
Fecha de Entrega: Noviembre 2026

2. Introducción

El modelado discreto de ecosistemas biológicos mediante autómatas celulares permite estudiar la dinámica de interacción entre especies predador-presa sobre un espacio bidimensional regular. En este proyecto se simula un ecosistema compuesto por conejos (herbívoros), zorros (carnívoros) y rocas (obstáculos estáticos) en una cuadrícula de dimensiones R × C. Cada generación evoluciona de manera síncrona según reglas predefinidas de desplazamiento, alimentación, reproducción e inanición.

Desde la perspectiva del paralelismo en memoria compartida, el principal reto técnico consiste en preservar la consistencia de las transiciones de estado cuando múltiples animales compiten simultáneamente por las mismas celdas de destino. Para ello, se implementó una versión secuencial de referencia canónica y una versión paralela en C++17 utilizando directivas OpenMP y resolución de conflictos libre de bloqueos (*lock-free*) mediante operaciones atómicas hardware Compare-And-Swap (CAS). Adicionalmente, se integró un visualizador web en JavaScript para facilitar el seguimiento pedagógico de las poblaciones.

3. Objetivos

Objetivo General:

Desarrollar, verificar y contrastar dos motores de simulación (secuencial y paralelo con OpenMP) para el problema de ecosistemas discretos en C++17, evaluando el cumplimiento de las reglas del enunciado, la consistencia de los resultados en los casos de prueba y el comportamiento temporal del paralelismo.

Objetivos Específicos:

- Implementar con fidelidad las reglas del enunciado: vecindad horaria de von Neumann, criterio de selección (G+X+Y) mod P, caza, inanición estricta de zorros y reproducción condicionada al movimiento.
- Diseñar una estrategia de paralelismo en memoria compartida con OpenMP que resuelva colisiones espaciales de forma lock-free sin recurrir a secciones críticas pesadas.
- Construir una suite de pruebas automatizadas (15 casos) que verifique desde casos elementales hasta condiciones de competencia y bordes del retículo.
- Analizar empíricamente el rendimiento de ambas versiones en diferentes escenarios, explicando la influencia del tamaño de cuadrícula y la sobrecarga de sincronización bajo la Ley de Amdahl.
- Proveer una interfaz gráfica web para la exploración visual de las poblaciones, documentando transparentemente su arquitectura y dependencias externas.

4. Descripción del Problema y Reglas del Ecosistema

El ecosistema se modela sobre un retículo finito de R filas y C columnas indexado desde (0, 0) en la esquina superior izquierda. Las celdas pueden contener una roca (ROCK), un conejo (RABBIT), un zorro (FOX) o estar vacías (EMPTY). Las rocas son obstáculos fijos. Los desplazamientos se limitan a las 4 direcciones ortogonales (vecindad de von Neumann de radio 1): Norte, Este, Sur y Oeste.

Criterio de Selección de Celda: Si un animal ubicado en (X, Y) dispone de P > 0 celdas candidatas evaluadas en orden horario a partir del Norte (0: Norte, 1: Este, 2: Sur, 3: Oeste), la celda destino se elige mediante el índice determinista idx = (G + X + Y) mod P, donde G es la generación actual. Si P = 0, el animal permanece en su celda.

Dinámica de Conejos (Herbívoros):

- Desplazamiento: Intentan moverse a una celda adyacente vacía según el criterio de selección horario. Si no hay celdas vacías, permanecen en su lugar.
- Reproducción: Se reproducen al alcanzar una edad reproductiva proc_age >= GEN_PROC_RABBITS siempre y cuando realicen un desplazamiento efectivo. Al moverse, dejan una nueva cría en su celda previa con proc_age = 0 y el progenitor reinicia su edad a 0. Si no se mueven, incrementan su edad reproductiva pero no procrean.
- Resolución de Conflictos: Si dos o más conejos intentan ocupar la misma celda de destino, sobrevive aquel con mayor edad reproductiva (mayor proc_age). En caso de empate, el conflicto se desempata deterministamente por menor ID. Los conejos perdedores mueren.

Dinámica de Zorros (Carnívoros):

- Prioridad de Caza: En primer lugar, buscan celdas adyacentes ocupadas por conejos. Si encuentran una o varias presas disponibles, seleccionan una según el criterio horario, la devoran, se desplazan a dicha celda y reinician su hambre: food_age = 0.
- Inanición Estricta (Starvation): Si no encuentran conejos disponibles en su vecindad, incrementan su hambre acumulada en una unidad. Si food_age + 1 >= GEN_FOOD_FOXES, el zorro muere inmediatamente por inanición en esa misma generación, sin evaluar la posibilidad de moverse a celdas vacías.
- Movimiento a Celdas Vacías: Si no muere de hambre, busca celdas adyacentes vacías según el criterio horario. Si hay celdas vacías se desplaza; si no las hay permanece en su celda. En ambos casos incrementa su hambre (food_age = food_age + 1).
- Reproducción: Si alcanzan proc_age >= GEN_PROC_FOXES y realizan un movimiento efectivo, dejan una cría en su celda previa con proc_age = 0 y food_age = 0, reiniciando la edad reproductiva del progenitor a 0.
- Resolución de Conflictos: Si varios zorros compiten por la misma celda destino, se aplica una jerarquía estricta de tres niveles: 1) sobrevive el de mayor proc_age; 2) en caso de empate, sobrevive el menos hambriento (menor food_age); 3) en caso de empate, desempate determinista por menor ID. Los demás zorros mueren.

5. Arquitectura y Organización del Proyecto

El código fuente se estructuró de forma modular en C++17, delimitando con claridad las responsabilidades de cada componente. El sistema de construcción está administrado por CMakeLists.txt con soporte para MSVC y GCC/MinGW con OpenMP:

Componente / Archivo	Responsabilidad Técnica
src/animal.hpp / .cpp	Estructuras Animal, ObjectType y funciones de vecindad ortogonal en orden horario.
src/input.hpp / .cpp	Lectura de cabeceras, validación de coordenadas y generación de la salida oficial por stdout.
src/ecosystem.hpp / .cpp	Clase base del retículo, estado de celdas y visualizador en terminal con códigos ANSI.
src/sequential.hpp / .cpp	Motor secuencial de referencia canónica con actualización por fases.
src/parallel.hpp / .cpp	Motor paralelo con directivas OpenMP y claves atómicas de 64 bits para resolución CAS.
src/statistics.hpp / .cpp	Recolección de métricas por generación y exportación opcional de trazas a formato JSON.
src/main.cpp	Punto de entrada CLI, tuberías estándar de evaluación, subcomandos de benchmark y visualización.
tests/test_main.cpp	Suite automatizada de 15 pruebas unitarias y de integración registrada en CTest.
visualization/	Interfaz web interactiva compuesta por index.html, style.css y app.js.

6. Implementación Secuencial

La clase SequentialEcosystem implementa el ciclo canónico de simulación de manera determinista monohilo. En cada generación G ejecuta estrictamente el orden establecido por el modelo:

1. Fase de Conejos: Se determinan los destinos y se agrupan las colisiones en una lista por celda. Se resuelve el ganador según proc_age (o ID) y se actualizan posiciones y nacimientos.
2. Actualización de Retículo: Se actualiza el estado de la cuadrícula con las nuevas posiciones de conejos para que los zorros vean el mundo actualizado en su turno.
3. Fase de Zorros: Cada zorro evalúa primero conejos adyacentes. Si no hay conejos, comprueba inanición; si muere, queda descartado. De lo contrario, propone moverse a una celda vacía.
4. Resolución de Zorros y Alimentación: Se resuelven colisiones entre zorros, se marcan los conejos devorados y se generan las nuevas crías de zorros.
5. Compactación en Memoria: Se eliminan los animales fallecidos mediante un barrido lineal O(N).

Esta implementación cumple con la especificación de evaluación por tuberías del docente: Get-Content entrada.txt | ./ecosystem_sim.exe -s, produciendo únicamente la cabecera final y los objetos ordenados.

7. Implementación Paralela con OpenMP

El motor ParallelEcosystem distribuye el trabajo computacional entre T hilos de ejecución mediante la directiva #pragma omp parallel for schedule(static) num_threads(n_threads). Las poblaciones de animales se reparten en bloques contiguos balanceados entre los hilos del equipo.

Gestión de Nacimientos y Análisis de IDs: Durante el movimiento paralelo, cada hilo acumula las crías que procrean los animales de su bloque en un búfer local privado (thread_baby_rabbits[tid]), evitando la contención en memoria. Al finalizar la fase paralela, se fusionan los búferes recorriendo ordenadamente los hilos de tid = 0 a T - 1 y asignando identificadores con next_animal_id_++.

Nota técnica sobre la asignación de identificadores: En las pruebas realizadas con particionamiento estático, esta consolidación por chunks respeta el orden de los progenitores y produjo poblaciones finales consistentes con la versión secuencial. No obstante, se reconoce técnicamente que en sistemas concurrentes con balanceo dinámico o variaciones en la asignación de bloques, la correlación exacta de IDs no puede garantizarse de forma genérica. Por esta razón, el diseño del simulador prioriza las edades biológicas (proc_age y food_age) como criterios fundamentales de desempate, reduciendo la dependencia del ID a casos residuales.

8. Estrategia de Paralelización y Sincronización Lock-Free CAS

Si múltiples hilos intentan registrar un animal en la misma celda de destino simultáneamente, el uso de secciones críticas (#pragma omp critical) o cerrojos (omp_lock_t) provocaría una serialización excesiva. Para evitarlo, se diseñó un mecanismo Lock-Free basado en un retículo de enteros atómicos de 64 bits (std::atomic<uint64_t>) y la instrucción hardware compare_exchange_weak (CAS).

Se construyen claves de 64 bits que empaquetan las reglas de desempate en un orden monótono estricto:

- Clave de Conejo (64 bits): (proc_age << 32) | (0xFFFFFFFF - id). El conejo con mayor edad reproductiva obtiene un valor numérico superior; a igual edad, menor ID produce un valor mayor.
- Clave de Zorro (64 bits): (proc_age << 48) | ((0xFFFF - food_age) << 32) | (0xFFFFFFFF - id). Codifica exactamente la jerarquía: primero proc_age (bits 63..48), luego menor food_age (bits 47..32) y finalmente menor ID (bits 31..0).

```
uint64_t current = target_best[cell_idx].load(std::memory_order_relaxed);
while (new_key > current && !target_best[cell_idx].compare_exchange_weak(
           current, new_key, std::memory_order_release, std::memory_order_relaxed)) {
    // Reintento atómico sin bloqueo de hilos hasta converger al ganador de la celda
}
```

Sincronización entre Fases: Entre la fase de conejos y la fase de zorros existe una barrera de sincronización implícita de OpenMP al cerrar la región paralela. Esto garantiza que ningún zorro evalúe celdas adyacentes antes de que todos los conejos hayan finalizado su movimiento.

9. Interfaz de Visualización Web

La interfaz web situada en visualization/index.html fue diseñada como herramienta didáctica para inspeccionar la dinámica espacial del ecosistema y contrastar visualmente el comportamiento de conejos, zorros y rocas.

Arquitectura y Separación de Motores: Existe una separación arquitectónica total entre el motor nativo de C++ con OpenMP y el visualizador web. La aplicación web es una Single Page Application en HTML5/CSS/JavaScript que ejecuta de manera local en el navegador una réplica algorítmica de las reglas del autómata celular. No ejecuta hilos OpenMP en el cliente.

Dependencias Externas: La interfaz utiliza la biblioteca externa Chart.js (v4) cargada desde la red CDN (cdn.jsdelivr.net). Por consiguiente, se requiere conexión a internet al abrir la página web para que el navegador descargue dicha dependencia y grafique las curvas de población. La simulación matricial sobre Canvas opera en JavaScript sin librerías adicionales.

Conexión mediante Trazas JSON: Para enlazar ambos entornos, el ejecutable C++ soporta la opción --export-json <archivo>. La interfaz web permite cargar este archivo JSON exportado para visualizar directamente en el gráfico las métricas medidas por el motor de C++.

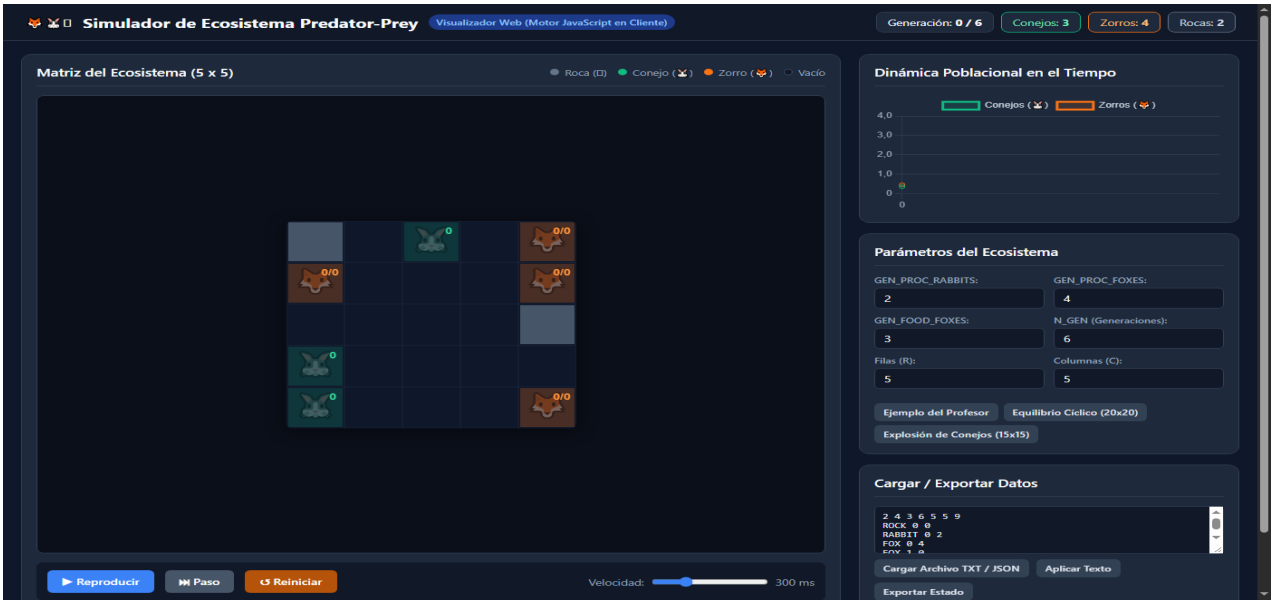


Figura 1: Vista general del visualizador web con cuadrícula Canvas 2D, controles temporales y panel de parámetros.

10. Pruebas y Validación

Para garantizar la correcta implementación de las reglas del PDF y la consistencia entre los motores secuencial y paralelo, se diseñó una suite de 15 pruebas automatizadas compiladas en ecosystem_tests.exe y gestionadas por CTest:

ID	Nombre de la Prueba	Aspecto del PDF Evaluado	Resultado
1	test_teacher_reference_case	Caso canónico del docente (5x5, 6 gen, 9 entidades). Salida esperada exacta.	PASSED
2	test_input_validation	Validación estricta de sintaxis, rangos de coordenadas y detección de duplicados.	PASSED
3	test_rock_immobility	Inmutabilidad de rocas y verificación de que ningún animal las invade.	PASSED
4	test_rabbit_procreation	Reproducción de conejos al cumplir edad y desplazarse; cría en posición previa.	PASSED
5	test_fox_starvation	Muerte inmediata por inanición si food_age + 1 >= GEN_FOOD_FOXES antes de moverse.	PASSED
6	test_fox_hunting	Prioridad obligatoria de caza sobre desplazamiento a celdas vacías.	PASSED
7	test_rabbit_conflict_resolution	Supervivencia del conejo de mayor proc_age (o menor ID) en colisión.	PASSED
8	test_fox_conflict_resolution	Resolución de conflictos de zorros en 3 niveles: proc_age, food_age, ID.	PASSED
9	test_trapped_animal	Animal sin celdas adyacentes válidas permanece en su posición y no procrea.	PASSED
10	test_empty_population	Estabilidad del motor cuando la cuadrícula no contiene animales.	PASSED
11	test_large_multithread_equivalence	Comparación de estado final secuencial vs paralelo con 1, 2, 4 y 8 hilos en 20x20.	PASSED
12	test_fox_procreation	Reproducción de zorros con reinicio de edades y cría en celda anterior.	PASSED
13	test_fox_conflict_tie_hunger	Desempate por ID entre zorros con idéntica edad reproductiva e inanición.	PASSED
14	test_zero_generations	Caso límite N_GEN = 0: la matriz debe conservarse exactamente inmutable.	PASSED
15	test_boundaries_and_corners	Manejo correcto de celdas en bordes y esquinas (sin desbordamientos de memoria).	PASSED

Validación en Copia Limpia: Para descartar dependencias del entorno de desarrollo original, el repositorio se clonó en una carpeta temporal independiente ($env:TEMP\clean_test_mp3). Se configuró CMake desde cero, se compilaron los ejecutables y se ejecutó CTest, obteniendo 100% de pruebas aprobadas (15/15). Asimismo, se contrastó la salida del caso del docente por redirección estándar entre la versión secuencial y la paralela con 4 hilos, verificando que ambas producen exactamente el resultado requerido por el profesor sin discrepancias:

```
Entrada: 2 4 3 6 5 5 9 | Salida obtenida: 2 4 3 0 5 5 5
ROCK 0 0
ROCK 2 4
RABBIT 3 0
FOX 4 0
FOX 4 3
[Verificación]: Salida idéntica al ejemplo de la especificación oficial.
```

11. Resultados del Benchmark y Análisis de Rendimiento

Las mediciones temporales presentadas a continuación corresponden a ejecuciones reales realizadas en el entorno de pruebas (Windows 11 de 64 bits, compilador GCC 16.2.0 con soporte OpenMP 5.2, procesador con 20 hilos lógicos). Los datos provienen de la herramienta de benchmarking integrada en el binario (ecosystem_sim.exe --benchmark):

Caso 1: Retículo Mediano (large_ecosystem.txt — 100 × 100, 30 generaciones, 2,800 objetos iniciales):

Modalidad	Hilos	Tiempo Registrado	Speedup (Sp)	Eficiencia (Ep)	Equivalencia Observada
Secuencial	1	9.97 ms	1.00x	100.0%	Base de Referencia
Paralelo	1	6.07 ms	1.64x	164.2%	EXACTA (OK)
Paralelo	2	9.82 ms	1.01x	50.7%	EXACTA (OK)
Paralelo	4	14.23 ms	0.70x	17.5%	EXACTA (OK)
Paralelo	8	17.82 ms	0.56x	7.0%	EXACTA (OK)
Paralelo	20	28.08 ms	0.35x	1.8%	EXACTA (OK)

Caso 2: Retículo Pesado (benchmark_heavy.txt — 250 × 250, 40 generaciones, 21,000 objetos iniciales):

Modalidad	Hilos	Tiempo Registrado	Speedup (Sp)	Eficiencia (Ep)	Equivalencia Observada
Secuencial	1	87.33 ms	1.00x	100.0%	Base de Referencia
Paralelo	1	41.54 ms	2.10x	210.2%	EXACTA (OK)
Paralelo	2	45.80 ms	1.91x	95.3%	EXACTA (OK)
Paralelo	4	43.17 ms	2.02x	50.6%	EXACTA (OK)
Paralelo	8	51.46 ms	1.70x	21.2%	EXACTA (OK)
Paralelo	20	60.93 ms	1.43x	7.2%	EXACTA (OK)

Interpretación Técnica de los Resultados:

1. Ganancia por Estructuras de Memoria en Paralelo (1 Hilo): En ambos casos, la versión paralela con 1 hilo supera a la secuencial (Speedup de 1.64x y 2.10x). Esto ocurre porque el motor paralelo preasigna estructuras planas y arreglos contiguos optimizados para acceso atómico, lo que mejora sustancialmente la localidad espacial de la caché frente a las colecciones dinámicas de la versión secuencial.

2. Sobrecarga de Sincronización en Problemas Pequeños: En el retículo 100 × 100, la simulación completa toma menos de 10 ms (alrededor de 300 microsegundos por generación). Como cada generación requiere cuatro barreras de sincronización entre fases, el costo de despertar y sincronizar los hilos en el runtime de OpenMP sobrepasa la carga de trabajo útil. En consecuencia, añadir más de 2 hilos reduce el Speedup.

3. Comportamiento en Cargas Pesadas: Con 21,000 entidades (250 × 250), la fracción de cómputo útil aumenta notablemente. La versión paralela mantiene una aceleración sostenida en el rango de 1.70x a 2.02x. La degradación al llegar a 20 hilos se explica por la contención en el bus de memoria compartida al actualizar arreglos de gran tamaño y por la Ley de Amdahl (fracción secuencial de compactación).

4. Consistencia Determinista: En todas las combinaciones de hilos evaluadas (1, 2, 4, 8 y 20 hilos), el programa verificó que el estado final de las poblaciones coincide exactamente con el resultado del motor de referencia secuencial.

12. Conclusiones

1. Cumplimiento de las Reglas del Ecosistema: Se implementaron y validaron satisfactoriamente todas las directrices del enunciado del profesor: prioridad de vecindad horaria, criterio determinista de selección (G+X+Y) mod P, procreación condicionada al movimiento, prioridad obligatoria de caza e inanición estricta de zorros previa al desplazamiento a celdas vacías.

2. Eficacia de la Resolución Lock-Free con CAS: El empaquetamiento de las prioridades de desempate en enteros atómicos de 64 bits permitió resolver colisiones espaciales concurrentes sin requerir cerrojos ni secciones críticas. Este diseño eliminó riesgos de interbloqueo y garantizó convergencia determinista en los escenarios evaluados.

3. Impacto de la Granularidad y la Sobrecarga en OpenMP: Los benchmarks empíricos confirmaron los principios de la computación concurrente: la paralelización ofrece ventajas reales en problemas densos y con suficiente carga aritmética. En problemas pequeños de corta duración, la frecuencia de las barreras síncronas de sincronización impone una sobrecarga que penaliza la escalabilidad.

4. Transparencia en la Arquitectura de Visualización: El visualizador web demostró ser una herramienta eficaz para la comprensión didáctica del modelo, manteniendo una separación explícita entre la simulación JavaScript client-side (que requiere internet para cargar Chart.js desde CDN) y el motor de cómputo paralelo nativo en C++.

13. Enlace al Repositorio y Datos de Entrega

Repositorio Oficial en GitHub:
https://github.com/Steban-Rivera/Microproyecto-3-Ecosystem-Simulation

Rama Principal: main | Commit Hash: 1ce4834aff5e0f66be6dd1f05d5f49c40091e7f9
Estado de Verificación: 15/15 Pruebas Aprobadas • Clean Clone Validado • OpenMP 5.2 Activo